

Project Submission Report: LearningSteps EvolutionStudent / Engineer: Nadeem Khan Repository:

nadeemkhan9876/learningsteps Project Window: July 6, 2026 – July 31, 2026 Architecture: 2-Tier Cloud-Native

Microservice (FastAPI + Azure Database for PostgreSQL) on Azure Kubernetes Service (AKS) 1. Executive

SummaryThe LearningSteps Evolution project transitions the legacy, hand-provisioned 2-tier application into

an automated, self-healing, and secure Cloud-Native ecosystem. Key AccomplishmentsInfrastructure as Code

(IaC): Defined and provisioned all Azure infrastructure (VNet, Subnets, AKS, Key Vault, PostgreSQL Flexible

Server, ACR) using modular Terraform configurations. Containerization: Packaged the FastAPI backend service

into a lightweight, production-grade Docker image stored in Azure Container Registry (ACR). Orchestration &

High Availability: Orchestrated containers on Azure Kubernetes Service (AKS) using Init Containers for

automated DB schema migrations, a LoadBalancer Service for ingress, and Horizontal Pod Autoscaling (HPA).

Automated DevSecOps & CI/CD: Engineered a GitHub Actions pipeline featuring static code analysis, IaC

security scanning, Docker image vulnerability checks, and automated deployments to AKS. 2. Architecture

Diagram+-----+ |

GITHUB REPOSITORY / CI/CD | | GitHub Actions (Build, Scan & Deploy) | +-----+ |

+-----+ | v +-----+ |

+-----+ | AZURE CONTAINER REGISTRY (ACR) | | acrlearningsteps123 | +-----+ |

+-----+ | v +-----+ |

+-----+ | VIRTUAL NETWORK:

learningstepsnk-vnet (10.0.0.0/16) | | | +-----+ +-----+

+-----+ | | | AKS SUBNET (10.0.0.0/22) | | DB SUBNET (10.0.4.0/24) | | | | | | | +-----+

+-----+ | | +-----+ | | | | Azure Kubernetes Service (AKS) | | | | Azure

Database for PostgreSQL | | | | - Pod Replicas (FastAPI App) |=|=|=|=| (learningstepsnk-pg:5432) | | | | -

Init Container (DB Setup) | | | +-----+ | | | | - HPA & LoadBalancer Service | | +-----+

+-----+ | | | +-----+ +-----+ | | | +-----+ |-----+

+-----+ | +-----+ |-----+

--+ | v Public Ingress Point <http://20.73.198.134:80> 3. Detailed Step-by-Step ImplementationStep 1: Application

Containerization (Dockerfile)The FastAPI application was containerized using a non-root, lightweight Python

image to enforce container security standards. Base Image: python:3.11-slim Exposed Port: 8000 Execution

Command: uvicorn app.main:app --host 0.0.0.0 --port 8000 DockerfileFROM python:3.11-slim WORKDIR /app

COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt COPY . . EXPOSE 8000 CMD

["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"] Step 2: Infrastructure as Code (/infra-

terraform)All required Azure resources were provisioned using modular Terraform code. Resource Group:

learningstepsnk-rg (Region: westeurope) Virtual Network: learningstepsnk-vnet (10.0.0.0/16) split into: aks-

subnet: 10.0.0.0/22 db-subnet: 10.0.4.0/24 (Delegated to Microsoft.DBforPostgreSQL/flexibleServers)

Compute: Azure Kubernetes Service cluster (learningstepsnk-aks) with Azure CNI networking. Secrets

Management: Azure Key Vault (kvevolution786) storing db-connection-string. PowerShell# Deployment

Commands cd infra-terraform terraform init terraform plan terraform apply -auto-approve Step 3: Kubernetes

Orchestration (/k8s-manifests)The application deployment utilizes an Init Container (init-db) to verify and seed

the PostgreSQL database schema prior to launching the main FastAPI API container. Key Resilience & Setup

Rules:Init Container Execution: Runs postgres:15-alpine to execute /scripts/database_setup.sql. Special

Character URL Encoding: Cleaned database passwords in Key Vault to ensure connection strings

(postgresql://user:pass@host:port/db) do not fail DNS resolution on reserved characters (@, !). High

Availability: Maintained a desired state of 2 replicas managed by a Deployment and Horizontal Pod Autoscaler

(HPA). Service Endpoint: Configured an external LoadBalancer Service mapping public port 80 to container

port 8000. YAMLaPiVersion: apps/v1 kind: Deployment metadata: name: learningsteps spec: replicas: 2

selector: matchLabels: app: learningsteps template: metadata: labels: app: learningsteps spec: initContainers: -

```
name: init-db image: postgres:15-alpine command: ["sh", "-c", "psql $DATABASE_URL -f
/scripts/database_setup.sql"] envFrom: - secretRef: { name: learningsteps-secrets } volumeMounts: - name: db-
script-volume mountPath: /scripts containers: - name: learningsteps image:
acrlearningsteps123.azurecr.io/learningsteps:latest ports: - containerPort: 8000 envFrom: - configMapRef: {
name: learningsteps-config } - secretRef: { name: learningsteps-secrets } volumes: - name: db-script-volume
configMap: name: db-init-script
```

Step 4: Securing the Pipeline (DevSecOps)

The deployment pipeline integrates automated security gates:

- Secret Protection:** Secrets are pulled directly from Azure Key Vault and passed into Kubernetes Secrets via environment variables without hardcoded file storage.
- Static Security Analysis:** Terraform scripts are scanned with checkov/trivy to block open ports or unencrypted storage configurations.
- Container Vulnerability Scanning:** Container images built in GitHub Actions undergo automated vulnerability scans prior to registry publication.

Step 5: Verification & Self-Healing Testing

- Pod Health Verification


```
PowerShellPS> kubectl get pods NAME READY STATUS RESTARTS AGE learningsteps-7d54669488-9f274 1/1 Running 0 18m learningsteps-7d54669488-bxbbn 1/1 Running 0 13s
```
- Self-Healing Demonstration

When a pod is deleted, the Deployment Controller detects the state mismatch (1/2 replicas) and immediately schedules a replacement pod:

```
PowerShellPS> kubectl delete pod learningsteps-7d54669488-z89w4 pod "learningsteps-7d54669488-z89w4" deleted
```

```
PS> kubectl get pods NAME READY STATUS RESTARTS AGE learningsteps-7d54669488-9f274 1/1 Running 0 18m learningsteps-7d54669488-bxbbn 1/1 Running 0 13s
```

- Public Service Access


```
PowerShellPS> kubectl get service learningsteps NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE learningsteps LoadBalancer 10.1.198.29 20.73.198.134 80:30621/TCP 51m Public Endpoint: http://20.73.198.134
```

4. Success Criteria

- Verification Criteria**
 - Status Implementation Details
 - Continuous Deployment PASSED
 - GitHub Actions automatically builds, scans, and deploys on git push.
 - Security Enforcement PASSED
 - Sanitized database credentials, zero hardcoded secrets, and Trivy scans.
 - Infrastructure Recovery PASSED
 - Environment fully reproducible via terraform apply.
- High Availability & Self-Healing PASSED**
 - Verified automatic pod replacement by AKS Deployment controller.